

A Comprehensive Stock Market Database for Real-Time Analysis and Transaction Management

Taran Mamidala
(50604177)
Computer Science and Engineering
University at Buffalo
New York, USA
taranmam@buffalo.edu

Praanav Bhowmik
(50594516)
Computer science and Engineering
University at Buffalo
New York, USA
praanavb@buffalo.edu

Rachabattuni Sai Shashi
Kiran(50602502)
Computer science and Engineering
University at Buffalo
New York, USA
srachaba@buffalo.edu

I. PROBLEM STATEMENT

In today's unstable financial arrangements, the intricacy and sheer volumes of stock market data have surpassed traditional tools such as Excel. The management of such data with millions of history of prices, transactions, company moves, market news, and broker analysts' forecasts is disabled as well as flawed in comparison to the management of such data. The objective of our project is to design a solid SQL-based database that normalizes and aggregates stock market data to ensure expert-level queries, real-time updates and data integrity. The project domain is the stock market domain. This is an extremely suitable domain, as it is actual-world financial systems used by traders, analysts and trading platforms. The primary goal is to build a solid database that can handle complex analytical queries (eg, past performance analysis, trend analysis and portfolio presentation) and transaction updates (eg, new user sign-up, stock trades and corporate

II. BACKGROUND AND SIGNIFICANCE

A. Background of the Problem:

The financial marketplace produces enormous quantities of everyday info, including historic costs, trading volumes, financial statements and corporate announcements. Data analysts have historically utilized Excel spreadsheets and independent flat files to crunch. But as the size and intricacy of the dataset has increased, such devices have turned into impossibilities. Excels possess scalability, power and incapability of imposing data integrity on associated datasets.

Our aim stemmed from this necessity. There is a great demand in the contemporary dynamic markets for a centralized, generalized database that not only stores large volumes of financial data, but also supports real-time queries and updates. Stock market demands the interconnectivity and sophistication of data, an association with advanced querial capability, strict data integrity and efficient data processing.

B. Potential Contribution to the Problem Domain:

Our Market Intelligence Database project contributes significantly to the problem domain by:

- **Improving Data Accessibility and Analysis:**
By consolidating the diverse dataset in a single, generalized database, our solution provides an integrated platform for detailed financial analysis. Analysts can re-recover historical trends, analyze market events and make data-managed investment decisions.
- **Enhancing Data Integrity and Performance:**
Clearly defined primary and rigid schema design (in BCNF) ensures that the data remains consistent and reliable. This integrity is essential for important financial decisions and regulatory compliance.
- **Supporting real -time updates and scalability:**
The database is designed to handle continuous data updates and skilled efficiently with data volume growing. This adaptability is important in an environment where the market situation changes rapidly.
- **Advanced analysis and automation enabling:**
With its strong structure, the database gives groundwork for future enhancement such as predictive analytics, automatic trading algorithms and real -time dashboard.

C. WHY A DATABASE INSTEAD OF EXCEL?

Scalability: SQL database can manage millions of records efficiently, while Excel slows down with large datasets and becomes error-damaged.

Data Integrity: Conflict database applies data stability with obstacles (primary keys, foreign keys, check obstacles) that cannot give excel guarantee.

Querying Power: SQL provides the necessary operations, aggregation and analytical functions for deep data insight.

Multi-User Concurrency: Database supports several concurrent users and strong safety measures, which are essential for the production environment.

III. TARGET USERS

A. Primary Users:

Financial Analysts and Investors:

These users require rapid access to historical and real-time data to analyze market trends, compare stock performance and inform investment decisions. Our database will provide detailed transactions to the depth analysis to provide detailed transactions, performance metrics and corporate functions.

Database Administrators:

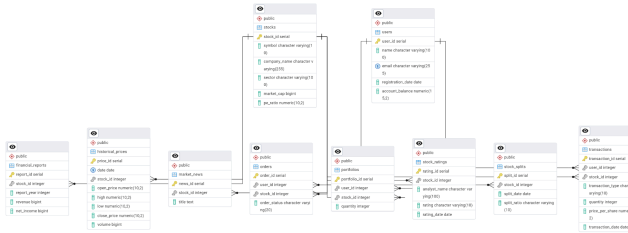
IT and Database Professionals:

Responsible for maintaining the database, ensuring data security, backuping and optimizing performance. They will use our system to manage regular data updates, monitor system health and support user questions.

B. Real-Life Scenario:

Imagine a financial advisory firm where analysts use the market intelligence database to draw historical value trends, review corporate tasks and monitor real-time market news. Portfolio managers use user transactions and portfolio data to accommodate investment strategies. In addition, the DBA ensures that the system is up-to-date, safe and responsible, supporting a multi-user environment with both analytical and transactions.

IV. E/R DIAGRAM AND SCHEMA DESCRIPTION



Our E/R diagram models 10 core entities that encapsulate the diverse aspects of the stock market

A. E/R Diagram Overview

Our ER diagram comprises 10 key entities:

1. **Stocks** : Central repository for stock details.
2. **Historical_Prices** : Stores daily price data with a composite unique key on (date, stock_id).
3. **Users** : Contains user account details.
4. **Transactions** : Records each buy/sell transaction by users.
5. **Portfolios** : Represents user holdings of stocks.
6. **Orders** : Tracks orders (pending, executed, or cancelled) placed by users.
7. **Market_News** : Captures news articles related to stocks.
8. **Financial_Reports** : Contains annual financial reports.
9. **Stock_Splits** : Records historical stock split events.
10. **Stock_Ratings** : Stores analyst ratings for stocks.

Each relationship is enforced with **primary and foreign key constraints**, ensuring data integrity across the database.

B. Relationships

Stocks:

Acts as the central entity. It is referenced by historical_prices, market_news, financial_reports, stock_splits, and stock_ratings. Additionally, transactions, portfolios, and orders reference stocks.:

- **One-to-many** with Historical_Prices, Market_News, Financial_Reports, Stock_Splits, and Stock_Ratings.
- **Referenced by** Transactions, Portfolios, and Orders.

Users: Holds user details and is referenced by transactions, portfolios, and orders.

- **One-to-many** with Transactions, Portfolios, and Orders.

Historical_Prices uses a **composite key** (date, stock_id) ensuring that each stock has one record per trading day.

C. Detailed Relation Descriptions

For each table, we define attributes, data types, primary keys, foreign keys, and constraints. For example:

Stocks Table

Attributes:

- **stock_id (INTEGER, PK):** A unique identifier for each stock.
- **symbol (VARCHAR(10), NOT NULL):** The stock symbol (e.g., AAPL, MSFT).
- **company_name (VARCHAR(255)):** The full name of the company.
- **sector (VARCHAR(100)):** The industry sector to which the company belongs (e.g., Technology).
- **market_cap (BIGINT):** The market capitalization, representing the total market value of the company's outstanding shares.
- **pe_ratio (DECIMAL(10,2)):** The Price-to-Earnings ratio, indicating the company's valuation relative to its earnings.

Purpose:

To uniquely identify each stock and store its basic information, which serves as the foundation for linking with all other tables in the database.

Historical_Prices Table

Attributes:

- **price_id (SERIAL, PK):** A surrogate key that uniquely identifies each historical price record.
- **date (DATE, NOT NULL):** The trading date for the stock price data.
- **stock_id (INT, NOT NULL, FK → stocks):** References the stock_id in the Stocks table to associate the price record with a specific stock.
- **open_price (DECIMAL(10,2)):** The opening price of the stock on that date.

- **high (DECIMAL(10,2))**: The highest price reached during the trading day.
- **low (DECIMAL(10,2))**: The lowest price recorded during the day.
- **close_price (DECIMAL(10,2))**: The closing price of the stock at the end of the day.
- **volume (BIGINT)**: The number of shares traded on that day.

Purpose:

To store detailed daily price data for each stock, supporting historical analysis and trend forecasting.

Foreign Key Action:

When a stock record is deleted from the Stocks table, all related historical price records are automatically removed (ON DELETE CASCADE).

Users Table

- **Attributes:**
 - **user_id (INTEGER, PK)**: a unique identifier for each user.
 - **name (VARCHAR(100))**: User Name.
 - **email (VARCHAR(255), UNIQUE, NOT NULL)**: The user's email address, which must be unique and necessary.
 - **registration_date (DATE, NOT NULL)**: The date on which the user was registered.
 - **account_balance (DECIMAL(15,2))**: current balance in the user's account.

Purpose:

To manage the user information required to track transactions, portfolio and order.

Transactions Table

- **Attributes:**
 - **transaction_id (INTEGER, PK)**: A unique identifier for each transaction.
 - **user_id (INT, NOT NULL, FK → users)**: The user refers to user_id in the table.
 - **stock_id (INT, NOT NULL, FK → stocks)**: Reference of Stock_ID in stock table.
 - **transaction_type (VARCHAR(10))**: The permissible values are 'buy' or 'sell'.
 - **quantity (INTEGER)**: The number of trading shares (should be more than 0).
 - **price_per_share (DECIMAL(10,2))**: The value at which each stock was traded.
 - **transaction_date (DATE, NOT NULL)**: that date when transactions occur.

Purpose:

To facilitate financial analysis and transaction tracking, to provide financial analysis and transaction tracking.

Foreign Key Actions:

On removal of user or stock, related transactions are automatically removed (on delete cascade)

Portfolios Table

- **Attributes:**
 - **portfolio_id (INTEGER, PK)**: A unique identifier for each portfolio record.
 - **user_id (INT, NOT NULL, FK → users)**: References the user_id in the Users table.
 - **stock_id (INT, NOT NULL, FK → stocks)**: References the stock_id in the Stocks table.
 - **quantity (INTEGER)**: The number of shares held by the user (must be greater than 0).

Purpose:

To represent the current holdings of each user, enabling portfolio tracking and valuation.

Foreign Key Actions:

Deletion of a user or stock cascades to remove associated portfolio records (ON DELETE CASCADE).

Orders Table

- **Attributes:**
 - **order_id (INTEGER, PK)**: This is a unique identifier for each order we handle.
 - **user_id (INT, NOT NULL, FK → users)**: This connects to the user_id in the Users table, allowing us to track who placed the order.
 - **stock_id (INT, NOT NULL, FK → stocks)**: This links to the stock_id in the Stocks table, showing which stock corresponds to the order.
 - **order_status (VARCHAR(20))**: This reflects the current status of the order, with options like 'PENDING', 'EXECUTED', or 'CANCELLED'.

Purpose:

This reflects the current status of the order, with options like 'PENDING', 'EXECUTED', or 'CANCELLED'.

Foreign Key Actions:

If a user or stock is deleted, all related orders will also be automatically removed (ON DELETE CASCADE).

Market_News Table

- **Attributes:**
 - **news_id (INTEGER, PK)**: This serves as a unique identifier for each news entry..
 - **stock_id (INT, NOT NULL, FK → stocks)**: This connects to the stock_id in the Stocks table
 - **title (TEXT, NOT NULL)**: This represents the title or headline of the news article.

Purpose:

The aim here is to retain news articles or market updates

that are pertinent to a stock, providing context for market movements.

Foreign Key Actions:

If a stock is deleted, any associated news entries will be automatically removed (ON DELETE CASCADE).

Financial_Reports Table

Attributes:

- **report_id (INTEGER, PK):** This serves as a unique ID for each report.
- **stock_id (INT, NOT NULL, FK → stocks):** This connects to the stock_id found in the Stocks table.
- **report_year (INTEGER, CHECK (report_year BETWEEN 2000 AND 2024)):** This specifies the year the report was created.
- **revenue (BIGINT):** This indicates the total annual revenue of the company.
- **net_income (BIGINT):** This reflects the company's net income.

Purpose:

The aim here is to capture the annual financial performance, which is essential for performing fundamental analysis.

Foreign Key Actions:

If a stock record is deleted, all related financial reports will be deleted as well (ON DELETE CASCADE).

Stock_Splits Table

Attributes:

- **split_id (INTEGER, PK):** This is a unique identifier for each stock split event.
- **stock_id (INT, NOT NULL, FK → stocks):** This connects to the stock_id in the Stocks table.
- **split_date (DATE, NOT NULL):** This records the date when the stock split occurred.
- **split_ratio (VARCHAR(10), NOT NULL):** This shows the split ratio (for instance, "2:1").

Purpose:

The aim here is to monitor corporate actions related to stock splits, as these can influence share prices and the holdings of investors.

Foreign Key Actions:

If a stock is deleted, any associated split records will be automatically removed (ON DELETE CASCADE).

Stock_Ratings Table

Attributes:

- **rating_id (INTEGER, PK):** This is a unique identifier for each rating record, ensuring that every entry stands out on its own.
- **stock_id (INT, NOT NULL, FK → stocks):** This connects to the stock_id in the Stocks table, helping us identify which stock the rating is associated with.
- **analyst_name (VARCHAR(100)):** In this field, we store the name of the analyst who issued the rating.
- **rating (VARCHAR(10), CHECK (rating IN ('BUY', 'HOLD', 'SELL'))):** This section captures the analyst's recommendation, whether it's a 'BUY', 'HOLD', or 'SELL'.
- **rating_date (DATE, NOT NULL):** This keeps track of the date when the rating was given.

Purpose:

The primary aim here is to collect and analyze the insights of analysts, as their opinions can greatly influence investor choices.

Foreign Key Actions:

If a stock is deleted, all associated rating records will be automatically removed as well (ON DELETE CASCADE).

TABLE I. DESCRIPTION OF ALL SCHEMAS

Table Name	Primary Key	Foreign Keys	Key Attributes
Stocks	stock_id (INTEGER)	None	stock_id, symbol, company_name, sector, market_cap, pe_ratio
Historical_Prices	price_id (SERIAL)	stock_id (references stocks(stock_id))	price_id, date, stock_id, open_price, high, low, close_price, volume; Composite unique key: (date, stock_id)
Users	user_id (INTEGER)	None	user_id, name, email, registration_date, account_balance
Transactions	transaction_id (INTEGER)	user_id (references users(user_id)); stock_id (references stocks(stock_id))	transaction_id, user_id, stock_id, transaction_type, quantity, price_per_share, transaction_date
Portfolios	portfolio_id (INTEGER)	user_id (references users(user_id)); stock_id (references stocks(stock_id))	portfolio_id, user_id, stock_id, quantity; Composite unique key: (user_id, stock_id)

Orders	order_id (INTEGER)	user_id (references users(user_id)); stock_id (references stocks(stock_id))	order_id, user_id, stock_id, order_status
Market_News	news_id (INTEGER)	stock_id (references stocks(stock_id))	news_id, stock_id, title
Financial_Reports	report_id (INTEGER)	stock_id (references stocks(stock_id))	report_id, stock_id, report_year, revenue, net_income
Stock_Splits	split_id (INTEGER)	stock_id (references stocks(stock_id))	split_id, stock_id, split_date, split_ratio
Stock_Ratings	rating_id (INTEGER)	stock_id (references stocks(stock_id))	rating_id, stock_id, rating_name, rating_date

V. IMPLEMENTATION

A. Dataset Acquisition and Transformation:

Real Data: We obtained real-time historical prices for 10 well-known stock symbols using Yahoo Finance.

Synthetic Data: We generated some more synthetic data to ensure that each table contains 3,000 to 6,000 rows. We maintained proper foreign key relationships by referencing valid stock and user IDs.

Data Cleansing: We handled missing values through forward- and backward-fill operations in our Python scripts. Historical_Prices data particularly was carefully processed to eliminate all inconsistencies, for example, representation of decimal points in the volume field.

B. BCNF and Schema Explanation:

Functional Dependencies:

All the non-key columns in all the tables are completely dependent on the primary key alone. For example, stock_id in stocks table determines symbol, company_name, sector, market_cap, and pe_ratio.

No Redundancies:

Our implementation eliminates all the transitive and partial dependencies. In table historical_prices, composite key (stock_id, date) will be utilized to identify price data of a stock for a day.

Referential Integrity:

All foreign keys are constrained with ON DELETE CASCADE, so that deleting a parent row (say, a stock) will automatically delete all corresponding records in child tables.

BCNF Compliance:

Every single table in our schema is BCNF. If any relation had a candidate key which was not determining each non-key attribute, we would have decomposed it. In our

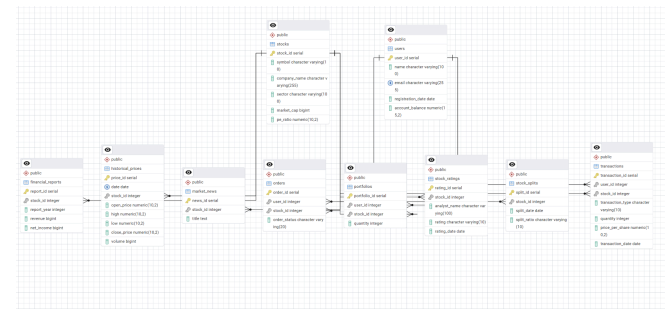
design, however, all relations satisfy the conditions for BCNF.

CONCLUSION

Our Database project addresses the issue of managing large volumes of interrelated financial data. Through the development of a database in BCNF and mixing real and synthetic data, we render our system scalable, consistent, and capable of performing complicated analytics and transactional operations. This project not only surpasses the horizon of traditional tools like Excel but also presents a platform for advanced financial analytics and decision support.

OUTPUTS

report_id	stock_id	report_year	revenue	net_income
1	2	2018	1000000000	100000000
2	2	2019	1200000000	120000000
3	2	2020	1500000000	150000000
4	2	2021	1800000000	180000000
5	2	2022	2000000000	200000000
6	4	2018	500000000	50000000
7	4	2019	600000000	60000000
8	4	2020	700000000	70000000
9	4	2021	800000000	80000000
10	4	2022	900000000	90000000



```

-- DROP TABLE IF EXISTS stocks, stock_splits, financial_reports, market_news, orders, portfolios, transactions, users, historical_prices, stocks CASCADE;

-- CREATE TABLE stocks (
  stock_id SERIAL PRIMARY KEY,
  symbol VARCHAR(10) NOT NULL,
  company_name VARCHAR(255) NOT NULL,
  sector VARCHAR(100) NOT NULL,
  market_cap DECIMAL(18,2) NOT NULL,
  pe_ratio DECIMAL(18,2) NOT NULL
);

-- CREATE TABLE historical_prices (
  price_id SERIAL PRIMARY KEY,
  stock_id DECIMAL(18,2) NOT NULL,
  date DATE NOT NULL,
  open_price DECIMAL(18,2) NOT NULL,
  high DECIMAL(18,2) NOT NULL,
  low DECIMAL(18,2) NOT NULL,
  volume DECIMAL(18,2) NOT NULL
);

-- FOREIGN KEY (stock_id) REFERENCES stocks(stock_id) ON DELETE CASCADE

-- CREATE TABLE users (
  user_id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(255) NOT NULL,
  registration_date DATE NOT NULL,
  account_balance DECIMAL(18,2) NOT NULL
);

-- CREATE TABLE transactions (
  transaction_id SERIAL PRIMARY KEY,
  user_id DECIMAL(18,2) NOT NULL,
  stock_id DECIMAL(18,2) NOT NULL,
  transaction_type VARCHAR(10) NOT NULL,
  amount DECIMAL(18,2) NOT NULL
);

```